

Every Student Gets the Same Classroom

github.com/seancnc003/UNLVdockerIDES
hub.docker.com/r/seancnc

Decentralized, browser-based Docker IDEs for C++ and x86-64 assembly at UNLV, measured on the machines students actually own

Sean Cuenco · Faculty Mentor: James Andro-Vasko, Ph.D. · Department of Computer Science, University of Nevada, Las Vegas · Office of Undergraduate Research SURF 2026

The problem

Every systems course loses its first week to installing a compiler, an assembler, a debugger, and an editor on dozens of different laptops. Prior work measured 75–100 min of class time and 65–90 min of instructor time per semester lost to installation [6], and support traffic dominated by environment mismatch rather than course content [3]. Students with M1/M2 Macs often cannot run the x86 virtual machines their courses assume at all [4].

UNLV followed the same arc, one stage behind

- Shared login servers (SSH / PuTTY)**
bobby · sally · cardiac, a submit script that required logging in even from lab machines. Decommissioned; the wiki that documented them was deleted in 2025.
- Virtual machines, per course**
CS 218 today: a VirtualBox .ova for Intel and an experimental Fall-2023 UTM image for Apple Silicon so slow the course warns VS Code “may not be usable.” Primary M1/M2 advice: go to the TBE-A311 lab with a USB drive.
- Local containers (this project)**
One 552 MB image, one command, VS Code in a browser tab. Files stay in a normal folder on the student’s disk. No server, no account, no VPN, offline after the first pull.

Where this sits in the design space

- Harvard CS50 [1, 2]**
Cluster → cloud VMs → client-side appliance VM → Docker on GitHub Codespaces (700,000+ users). Browser VS Code, but the container runs on GitHub: account, network, provider dependence; students leave “not sure how to set up an IDE on my computer.”
- CodeDive, South Korea [8]**
Browser VS Code in per-student containers on a Kubernetes cluster, with kernel-level process tracing and a code snapshot after every 1 s typing pause (24,845 snapshots from 95 students in two weeks). Visibility for instructors, at the cost of telemetry and a GDPR/FERPA framework the authors have not yet built.

- Virtual machines [5, 6, 7]**
Reading’s QEMU ARM emulator raised lab marks over six cohorts, but changed the course’s instruction set to keep gdb working. West Point’s server-hosted VMs halved class time lost to installs, but faculty ran the server fleet.

- Local containers [3, 4]**
UCSD DevContainers (71% chose them over the campus server) and Florida Tech (84% adoption, 69% less anxious). Both found the hardest step is installing Docker itself, and novices struggle most.

UNLV Docker IDEs

Local container + browser VS Code + host-owned files + no telemetry + AI surfaces disabled + real x86-64 ISA preserved, with the emulation boundary measured instead of ignored.

The system

Two Docker images built from Ubuntu 22.04 with code-server 4.126.0 (VS Code served to a browser on 127.0.0.1). CS 135: g++ 11.4, multi-arch, native on Intel and ARM. CS 218: yasm, nasm, ld, gdb 12.1, deliberately amd64-only so .asm files assemble and run as real x86-64 Linux binaries; ARM hosts run it under translation. Autosave, starter files seeded once, no restart policy, every AI feature disabled.

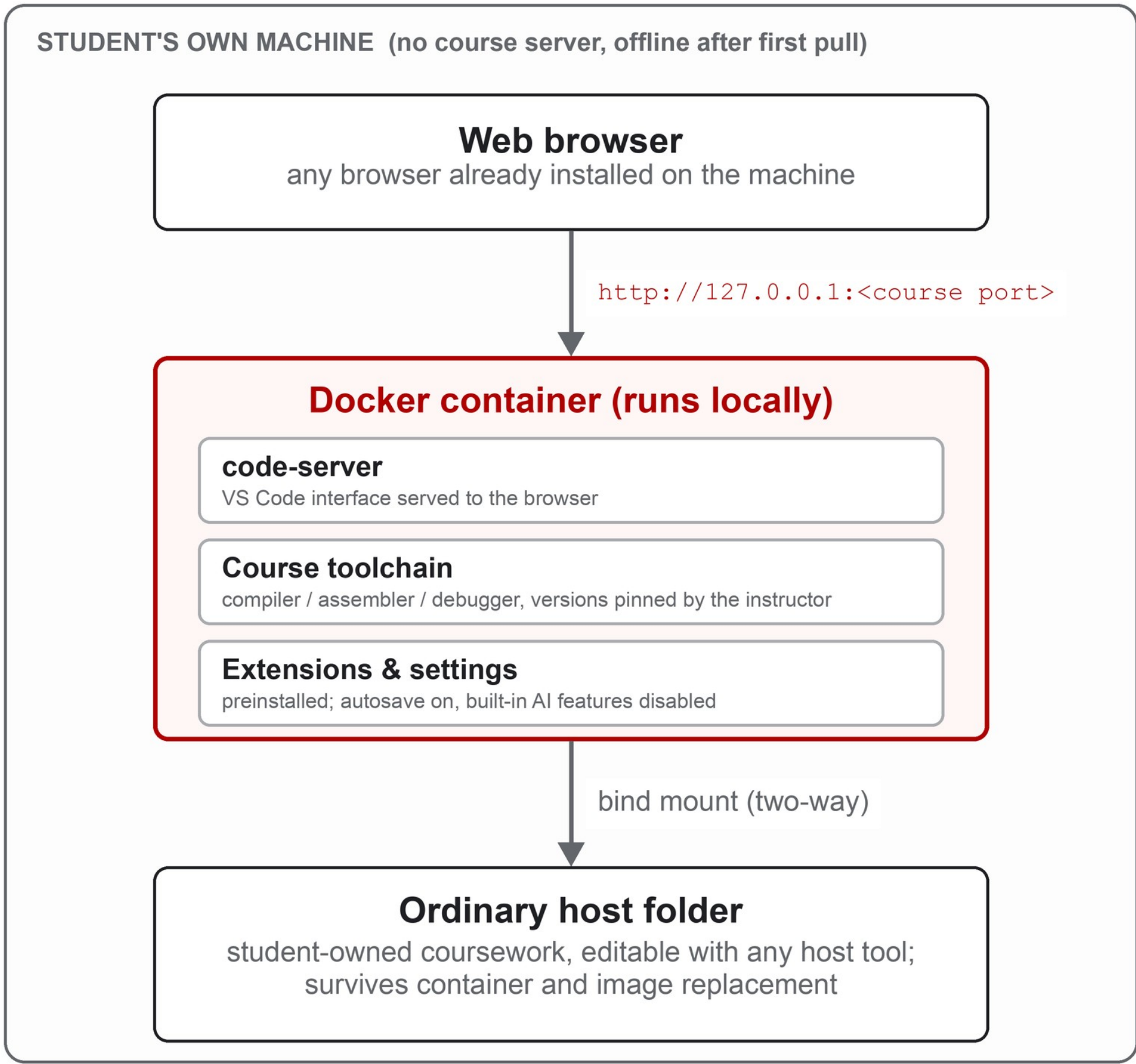


Figure 1. One course IDE. Everything runs on the student’s machine; the bind-mounted folder survives container and image replacement.

Method: one script, four machines

One published script (ci-test.sh) runs unmodified on every host and emits a JSON row; every number here comes from those files. Nine stages: timed pull and digest · cold start to healthy · starter seeding · 3× assemble-link-run · scripted gdb probe · idle memory · tool versions · build + run of four real CS 218 assignments (ast3, ast04, ast06, ast12: pure assembly through multithreaded pthread + assembly with a checkable answer) with peak memory · destroy-and-recreate to test persistence and warm start.

Cell	Host	x86 image
1 Linux amd64	AWS m8i.large · Ubuntu 24.04 · Intel Xeon 6975P-C · 2 vCPU / 8 GB	native
2 Linux arm64	AWS m8g.large (Graviton) · Ubuntu 24.04 · Neoverse-V2 · 2 vCPU / 8 GB	emulated (QEMU user-mode)
3 Windows amd64	Azure D4s_v5 · Windows 11 Pro 24H2 · Docker Desktop + WSL2 · 4 vCPU / 16 GB	native
4 macOS arm64	MacBook Pro 14" (2021), owned · M1 Pro · 16 GB · Docker Desktop	emulated (Rosetta)

Cloud machines were provisioned by hand from written runbooks and destroyed after each run: three of the four student environments summoned for cents to about a dollar each. Cells 1 and 2 are same-size Intel/Graviton siblings, so their ratio isolates the emulation layer. Cell 4 is the base-spec M1 Pro, not the 64 GB development machine, because that is what a budget-conscious student buys.

Results

4 / 4

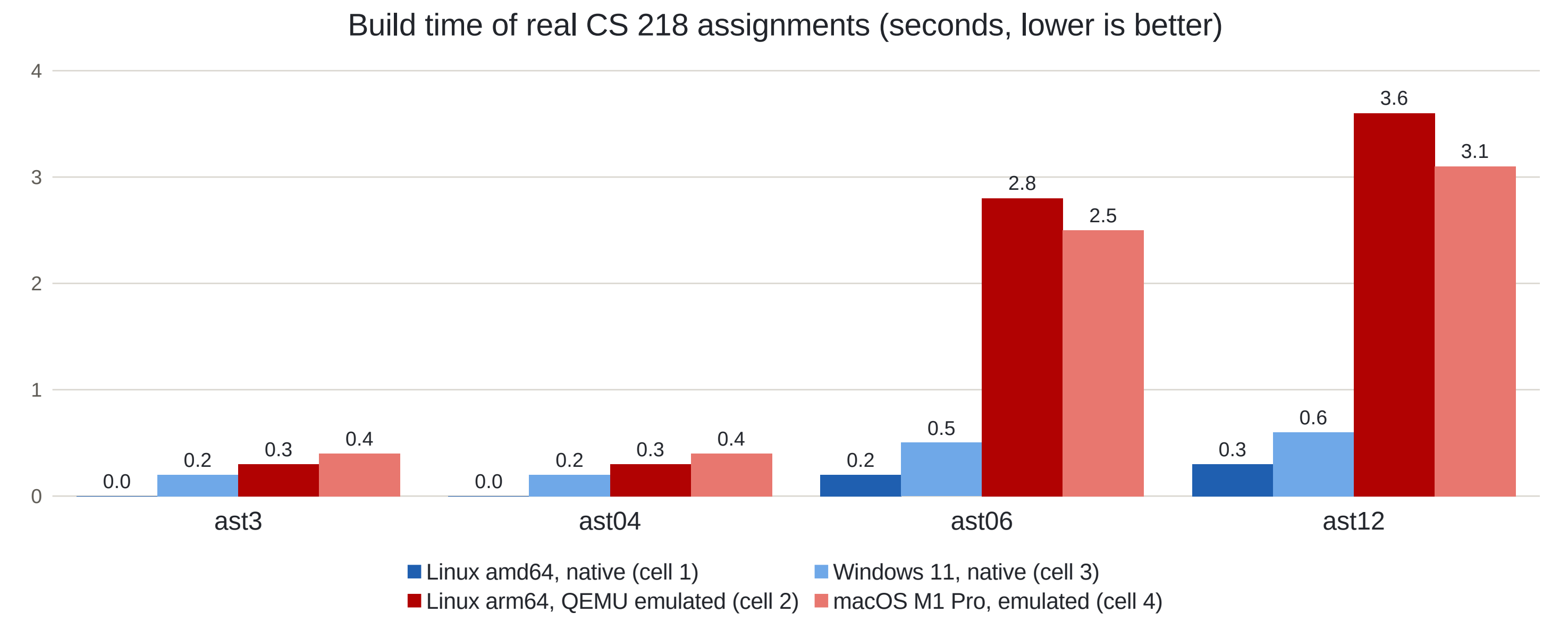
hosts ran every CS 218 assignment correctly, incl. multithreaded ast12 under emulation

1 digest

same 552 MB image, byte for byte, on Linux, Windows, and macOS

0.5 s

editor ready on native hosts; 5–7 s under emulation; 54 MiB idle native



Emulation costs roughly one order of magnitude on compile-heavy steps (ast06: 14× on QEMU, 12.5× on Docker Desktop; ast12: 12× and 10×), yet every emulated build finished in under 4 s. Peak memory during coursework: 90 MiB native, 334–358 MiB emulated, a small slice of an 8 GB laptop.

What it runs on, and where it stops

Host	Assemble / run	gdb	Evidence
Windows or Linux on Intel / AMD	works	works	cells 1, 3: 13/13 checks
Apple Silicon Mac	works	broken	cell 4: 12/12, gdb probe silent
ARM Linux	works with Docker’s binfmt handler	broken	cell 2: stock QEMU 8.2.2 crashed (SIGSEGV in the editor’s JIT), reproduced, passed 12/12 after switching handler
Windows on ARM	expected like cell 4	expected broken	untestable in any cloud: no nested virtualization on ARM VMs
Intel Mac	expected native	expected works	untested; no hardware

The boundary is a translator boundary, not an OS boundary. The image runs wherever the layer hosting Docker’s Linux environment needs no translation or receives a Rosetta-class one. Under translation a gdb script still “succeeds” and writes its file, but with labels and no register values, so the handouts tell students to produce debugger deliverables on a native x86 machine.

References

[1] Malan. Containerizing CS50. ITICSE 2024.
[2] Malan. From Cluster to Cloud to Appliance. ITICSE 2013.
[3] Valstar, Griswold, Porter. Using DevContainers to Standardize Student Development Environments. ITICSE 2020.
[4] Fernalld, OConnor, Sudhakaran, Nur. Lightweight Symphony. SIGITE 2023.
[5] Cadenas et al. Virtualization for Cost-Effective Teaching of Assembly Language Programming. IEEE Trans. Educ. 2015.
[6] Harvie, Cody, Morrell, Estes. Using Virtual Machines to Enhance the Educational Experience. SIGITE 2019.
[7] Laadan, Nieh, Viennot. Teaching OS Using Virtual Appliances. SIGCSE 2010.
[8] Park et al. CodeDive: A Web-Based IDE with Real-Time Code Activity Monitoring. Applied Sciences 2025.

Next

- Classroom study in CS 218 and CS 135 (IRB): setup-success funnel, staff minutes, forum traffic, pre/mid/post anxiety and setup-difficulty items adapted from [3, 4, 6]. A pilot in a 3-student high-school class ran without metrics.
- Hybrid arm64 assembly image: editor and toolchain native on ARM, x86 cross-assembler emits real x86-64 binaries, translator runs only student programs, gdb via QEMU’s built-in stub. One variant covers ARM Linux and Windows-on-ARM.
- Close the two untested cells with physical hardware; reuse the one-script, cloud-rented matrix for other courses (e.g., CS 370).